Многопоточная модель экзамена

Индивидуальное задание № 2. Вариант 17

«Архитектура компьютера и операционные системы»
НИУ ВШЭ, факультет компьютерных наук

Исполнитель: Бадретдинов Азат Рустэмович
Группа: БПИ252

Москва, 2026

Отображение участников в потоки

Создаются S потоков студентов и P потоков преподавателей, включая студентов, которым не достался билет. Главный поток создаёт ресурсы, подготавливает случайные входные данные, запускает участников и управляет остановкой. Всего S + P + 1 потоков ОС. Билеты, работы и очередь являются пассивными данными.

Случайные параметры вычисляются главным потоком заранее. Поэтому генератор не является совместно изменяемым объектом; одинаковый seed фиксирует входы, а порядок реальных действий потоков остаётся недетерминированным.

Поток студента:

1. Ожидает открытия общего стартового семафора.
2. При отсутствии билета сообщает об отказе и завершается.
3. Сообщает билет, переходит в PREPARING и выполняет задержку подготовки.
4. Переходит в READY, резервирует свободное место семафором slots.
5. Под мьютексом помещает индекс своей работы в FIFO, переходит в QUEUED, увеличивает items и освобождает мьютекс.
6. Ожидает личный семафор result; читает собственную оценку под мьютексом, сообщает результат и завершает работу.

Поток преподавателя:

1. Ожидает общий старт, затем семафор items.
2. Под мьютексом извлекает старейшую переданную работу, переводит её в CHECKING, записывает свой номер, освобождает место через slots.
3. Проверяет работу вне критической секции, моделируя затраченное время.
4. Под мьютексом записывает оценку, переводит работу в DONE, обновляет статистику и будит только владельца через его result; затем принимает следующую работу.

Синхронизация и корректность

Один pthread_mutex_t защищает очередь, состояния и результаты студентов, статистику, флаг остановки и запись полных строк журнала. Он не удерживается при sem_wait или временной задержке. Поэтому студент, ожидающий места или результата, не мешает преподавателю извлечь работу и выдать оценку.

Семафор slots изначально равен вместимости очереди, items равен нулю. Студент резервирует слот до вставки; преподаватель получает разрешение items до извлечения. Работы не копируются: очередь содержит индексы, у каждого студента сохраняется единственная запись.

Личный result изначально равен нулю и исключает получение чужой оценки. Стартовый gate позволяет сначала создать всех участников.

Инварианты нормальной работы:

- Размер очереди находится между нулём и заданной вместимостью.
- Каждый студент передаёт работу и получает оценку не более одного раза.
- Преподаватель не принимает вторую работу до окончания первой.
- Извлечение строго следует порядку фактической постановки в FIFO.
- DONE означает записанную оценку 2..5 и номер проверявшего преподавателя.
- admitted = completed + unfinished; students = admitted + rejected.

Для всех ожидающих семафоров при остановке добавляются разрешения пробуждения. Они не обозначают новые работы: после пробуждения поток проверяет stop под мьютексом и завершает работу. После запроса остановки не создаются новые оценки. Если оценка уже готова, студент всё равно выводит её. После pthread_join главный поток печатает неизменяемые итоговые данные и уничтожает синхропримитивы.

Интерфейс, время и завершение

Программа имеет интерфейс командной строки. Параметры приведены в таблице ниже; ./exam –help выводит краткую справку. Без аргументов запускается пример с двумя преподавателями, шестью студентами и шестью билетами.

Параметр	Значение по умолчанию	Назначение
`--teachers N`	2	1..1000 преподавателей
`--students N`	6	0..10000 студентов
`--tickets N`	6	0..10000 уникальных билетов
`--ticket-mode M`	random	random / sequential
`--prep-min N`	2	Минимальная подготовка, 1..100000
`--prep-max N`	5	Максимальная подготовка, 1..100000
`--check-time N`	2	Время проверки, 1..100000
`--grade-mode M`	random	random / ticket
`--queue N`	3	1..10000 мест для переданных работ
`--max-time N`	100	Лимит, 0..100000; 0 без лимита
`--seed N`	17	1..2147483647
`--delay-ms N`	10	0..60000; 0 соответствует 1 мс
`--log PATH`	exam.log	Перезаписываемый журнал

Нельзя задавать `prep-min > prep-max`. Некорректный ввод возвращает код 2. Штатное завершение и лимит времени возвращают 0; SIGINT - 130, SIGTERM - 143; ошибки ресурсов и ввода-вывода - 2. Причина остановки указывается в строке ИТОГ.

Сборка и примеры запуска

```
cd /mnt/d/codex/Prj17/hw2
make
./exam --help
```

```
./exam --delay-ms 100 --max-time 0
make test
bash launch_scripts/showcase.sh
```

Для сборки нужны C11-компилятор и POSIX pthread; для тестов нужен Python 3.

```
./exam --teachers 1 --students 8 --tickets 8 --queue 1 \
  --prep-min 1 --prep-max 1 --check-time 5 \
  --max-time 0 --delay-ms 100
```

Ctrl+C запрашивает остановку. Файл exam.log перезаписывается при каждом запуске; –log PATH позволяет выбрать другой журнал.

Одна единица подготовки, проверки и лимита времени соответствует delay-ms миллисекундам (по умолчанию 10; при 0 используется 1 мс). Интервалы измеряются CLOCK_MONOTONIC. Длительные задержки разбиты на участки до 10 мс для реакции на остановку. Временные отметки в журнале измеряются в реальных миллисекундах. Предел времени мягкий: зависит от частоты проверки и планировщика ОС.

Штатное завершение наступает после выставления оценок всем допущенным. Нулевое количество студентов или билетов не приводит к ожиданию несуществующих работ. max-time=0 отключает ограничение; положительное значение ограничивает время от общего старта. При timeout незаконченные работы сохраняются в итоговых состояниях. Завершённость всех работ имеет приоритет над тайм-аутом при одной проверке главным потоком; точное совпадение событий на границе не гарантируется.

SIGINT и SIGTERM устанавливают volatile sig_atomic_t. Обработчик не выполняет ввод-вывод и не захватывает блокировки. Главный поток обнаруживает сигнал, выставляет защищённый stop и будит участников. При частичной ошибке создания потоков освобождаются только созданные ресурсы и присоединяются только успешно запущенные потоки. Код ошибки создания потока выводится через strerror.

Журнал открывается open; все события записываются write с обработкой EINTR и коротких записей. Терминал и журнал получают одни и те же строки. Для справки и диагностики используются printf/fprintf/perror. Файл по –log перезаписывается. Закрытие stdout через SIGPIPE на Linux превращается в ошибку записи и остановку. Итог содержит состояния всех работ, причину, допуск, завершённость, отказы, пиковую очередь, количество ожидавших студентов и среднюю оценку.

Проверки и фактические результаты

Сборка GCC с C11, -Wall -Wextra -Wpedantic -Werror -O2 и -pthread выполнена без предупреждений. В среде Windows/MinGW-w64 прошла 31 интеграционная проверка. Сводка сохранена в results/test-summary.txt, версия компилятора в compiler.txt, журналы успешных сценариев находятся в results/.

Проверка	Наблюдавшийся результат
Стандартный набор без лимита	6 допущены, 6 завершили, средняя оценка 3.17
Один преподаватель, 8 студентов, очередь 1	8 завершили, 7 ожидали место, средняя оценка 3.50

Проверка	Наблюдавшийся результат
4 преподавателя, 20 студентов	В журнале наблюдались одновременно начатые проверки разных работ
5 студентов, 2 билета	2 завершили, 3 отказа
0 студентов; 0 билетов	Штатное завершение без зависания
Тайм-аут во время подготовки	reason=timeout, незаконченные работы учтены
Тайм-аут с полной очередью	Ожидающие участники разбужены, процесс завершился
11 неверных наборов аргументов	Код возврата 2
10 нагрузочных запусков по 60 студентов	FIFO, вместимость, уникальность билетов и проверки соблюдены
Несуществующий каталог журнала	Код возврата 2

Тесты восстанавливают FIFO по событиям, контролируют текущую работу каждого преподавателя, соответствие оценки студенту, диапазон оценок, отсутствие повторных работ, баланс итогов и равенство stdout сохранённому журналу. Отдельно проверяется перекрытие времени нескольких проверок, а не только наличие потоков в коде. Тесты не требуют определённого расписания потоков.

Затраты и ограничения

Постановка и извлечение работы имеют сложность O(1). Подготовка входов требует O(S+B), память данных O(S+P+Q+B). Системные ресурсы дополнительно расходуются на S+P стеков потоков и S+3 семафора. Границы аргументов не обещают, что ОС позволит создать 10000 потоков: при исчерпании ресурсов возвращается ошибка.

Справедливость пробуждения семафоров не гарантируется; FIFO действует после фактической передачи работы. Порядок строк и среднее время могут меняться при неизменном seed. Медленный терминал или файл задерживает общую критическую секцию. Блокирующий вывод в зависшее устройство не имеет гарантированного времени остановки. Проверки не являются доказательством отсутствия всех гонок; ThreadSanitizer и длительные Linux-нагрузки в этой среде не запускались.

Использование ИИ и критическая оценка

ChatGPT использовался для подготовки автоматических тестов на Python и файла README с описанием запросов.

Запросы к ИИ

1. «Напиши тесты для моего проекта». Результат: автоматические тесты в `tests/test_exam.py`.
2. «Напиши мне readme файл с промтами». Результат: файл `README_AI.md`.